



SIFT: enhance the performance of vulnerability detection by incorporating structural knowledge and multi-task learning

Liping Wang¹ · Guilong Lu² · Xiang Chen² · Xiaofeng Dai¹ · Jianlin Qiu²

Received: 31 July 2024 / Accepted: 9 March 2025

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract

Software vulnerabilities pose significant risks to software systems, leading to security breaches, data loss, operational disruptions, and substantial financial damage. Therefore, accurately detecting these vulnerabilities is of paramount importance. In recent years, pre-trained language models (PLMs) have demonstrated powerful capabilities in code representation and understanding, emerging as a promising method for vulnerability detection. However, integrating code structure knowledge while fine-tuning PLMs remains a significant challenge. To alleviate this limitation, we propose a novel vulnerability detection approach called SIFT. SIFT extracts the code property graph (CPG) to serve as the source of graph structural information. It constructs a code structure matrix from this information and measures the difference between the code structure matrix and the attention matrix using Sinkhorn Divergence to obtain the structural knowledge loss. This structural knowledge loss is then used alongside the cross-entropy loss for vulnerability detection in a multi-task learning framework to enhance overall detection performance. To evaluate the effectiveness of SIFT, we conducted experiments on three vulnerability detection datasets: FFmpeg+Qemu, Chrome+Debian, and Big-Vul. The results demonstrate that SIFT outperforms nine state-of-the-art vulnerability detection baselines, achieving performance improvements of 1.74%, 10.19%, and 2.87% in terms of F1 score, respectively. Our study shows the effectiveness of incorporating structural knowledge and multi-task learning in enhancing the performance of PLMs for vulnerability detection.

Keywords Vulnerability detection · Fine-tuning · Pre-trained language model · Structural Knowledge injection · Multi-task learning

Liping Wang, Guilong Lu have contributed equally to this work.

Extended author information available on the last page of the article

1 Introduction

Software vulnerabilities refer to errors during the software design, development, or deployment process, which may lead to unexpected behavior under specific conditions, resulting in security risks or system failures (Aslan et al. 2023). The impact of software vulnerabilities can be profound. For instance, in March 2023, researchers discovered a security vulnerability in Microsoft Outlook, CVE-2023-23397 (Hadipour and Murimi 2024), which allowed hackers to send specially crafted emails that would automatically trigger when retrieved and processed by the Outlook client, causing the victim to connect to an attacker-controlled external UNC location. This vulnerability was exploited on a large scale by hacker groups, including APT28, for up to a year, causing incalculable damage.

Recently, pre-trained language models (PLMs) trained on large-scale corpora have demonstrated significant effectiveness across various natural language and software engineering tasks (Li et al. 2023; Zheng et al. 2023). The domain of vulnerability detection is no exception. Researchers have proposed various methods based on PLMs to enhance the performance of vulnerability detection. For instance, Hanif and Maffeis (2022) pre-trained a custom tokenization pipeline on real-world C/C++ code to train a RoBERTa model. Subsequently, this model was fine-tuned for vulnerability detection tasks. Similarly, Fu and Tantithamthavorn (2022) constructed a BERT architecture-based model, pre-trained it, and fine-tuned it for vulnerability detection at the line level.

However, current PLM-based vulnerability detection methods suffer from a significant limitation: the ineffective integration of code structure knowledge during fine-tuning. We categorize existing PLMs into two distinct categories: PLMs with code structural information (structured models) and PLMs without code structural information (nonstructural models), such as CodeBERT (Feng et al. 2020), treat code fragments as token sequences, disregarding the inherent structural characteristics of the code. In contrast, structured models, such as CodeT5 (Wang et al. 2021) and GraphCodeBERT (Guo et al. 2020), aim to integrate more code structure information into the model. Unfortunately, even structured models only incorporate some structure-related training tasks during pre-training to acquire general structured knowledge, which is unrelated to downstream tasks. For instance, Wang et al. (2021) proposed the CodeT5 model (categorized as a structured model), they introduced two additional tasks: identifier tagging and masked identifier prediction, which mainly involve the model predicting whether a code token is an identifier. There is a natural gap between these tasks and the fine-tuning task of vulnerability detection. Therefore, we aim to incorporate domain-specific code structure knowledge of downstream tasks into the model through fine-tuning. However, how to effectively enhance structural knowledge during the fine-tuning stage has not been thoroughly investigated.

In this study, we propose SIFT to address the limitation. In existing studies, Zugner et al. (2021) tried incorporating structural knowledge by modifying the attention mechanism within the Transformer blocks, but this method involves extensive model modifications and is not universally applicable to pre-trained

models. Inspired by Kalyan et al. (2021), we propose training the model using a multi-task learning (MTL) approach. MTL allows the model to learn useful knowledge across tasks. As shown in Fig. 1, we use the original cross-entropy loss of the vulnerability detection task and the newly designed structural knowledge loss to perform multi-task learning, effectively injecting structural knowledge into the model and thereby enhancing its performance.

To evaluate the performance of SIFT, we utilize three widely studied benchmark datasets for software vulnerability detection: FFmpeg+Qemu (Zhou et al. 2019), Chrome+Debian (Chakraborty et al. 2022) and Big-Vul (Fan et al. 2020). We comprehensively compare nine state-of-the-art (SOTA) software vulnerability detection baseline approaches, including three token-based approaches (i.e., VulDeePecker (Li et al. 2018), Russell et al. (2018)), SySeVR (Li et al. 2022), three graph-based approaches (i.e., Devign (Zhou et al. 2019), Reveal (Chakraborty et al. 2022), and AMPLE (Wen et al. 2023)), and three LLM-based approaches (i.e., LineVul (Fu and Tantithamthavorn 2022), CodeBERT (Feng et al. 2020), and GRACE (Lu et al. 2024)). The comparison results demonstrate that our approach SIFT outperforms all baseline approaches. In particular, SIFT achieves performance improvements of 1.74%, 10.19%, and 2.87% in terms of F1 score, respectively.

In summary, the main contributions of our work can be summarized as follows:

- We propose a novel approach called SIFT, which integrates structural knowledge of vulnerability code during the fine-tuning process and employs a multi-task learning strategy. This method effectively addresses the insufficiency of absorbing code structure information during fine-tuning.

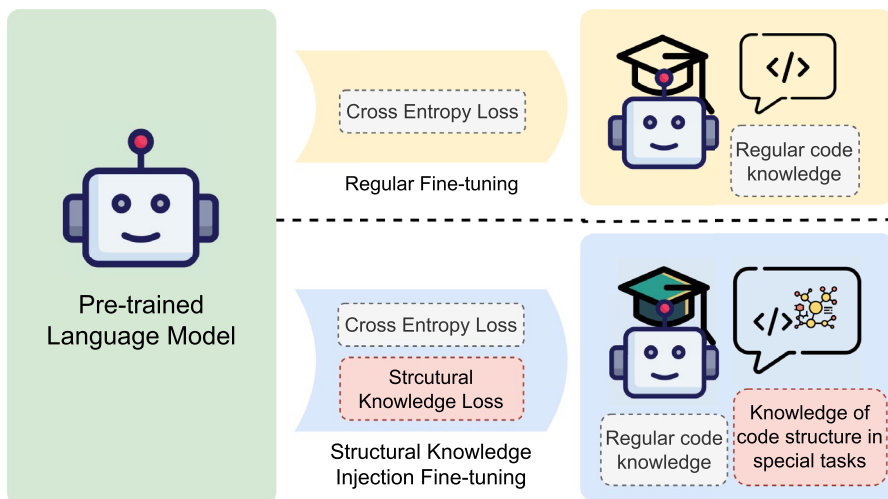


Fig. 1 Compared to regular fine-tuning, PLMs can effectively capture task-specific code structure knowledge by using our proposed approach SIFT

- We propose the concept of structural knowledge loss, which quantifies the difference between the attention matrices and the code structure matrices using Sinkhorn divergence.
- We perform a large-scale evaluation of SIFT on three public benchmark datasets, and the results demonstrate SIFT can achieve better performance compared to SOTA DL-based models for vulnerability detection.

Open science. Our source code and experimental data are available at our GitHub project homepage.¹

Paper organization. Section 2 introduces the background and research motivation. Section 3 presents the overall architecture of SIFT and the detailed methodology, including data preprocessing, graph structure information generation, and multi-task learning. Section 4 describes the experimental setup, including datasets, baselines, metrics, and experimental settings. Section 5 introduces the experimental results and analysis. Section 6 compares SIFT with two other tuning-based methods (fine-tuning and prompt tuning), analyzes the impact of the hyperparameter ω on SIFT's performance, and discusses the potential threats to validity. Section 7 analyzes related work, and Sect. 8 concludes this paper.

2 Background and motivation

2.1 Vulnerability detection based on PLMs

Pre-trained language models (PLMs) have significantly improved the performance of various natural language processing (NLP) and software engineering (SE) tasks, such as machine translation (Han et al. 2022) and code understanding (Karmakar and Robbes 2021), vulnerability detection is no exception. These models typically undergo a two-step process: pre-training and fine-tuning. ❶ Pre-training: In this phase, the model is trained on a large-scale, unlabeled corpus using self-supervised learning techniques. The objective is to learn general vector representations that capture the underlying patterns and structures within the data. For code, this might involve learning to predict masked tokens, understand code syntax, and capture semantic relationships. ❷ Fine-tuning: After pre-training, the model is fine-tuned on a smaller, labeled dataset specific to the target task. In the context of vulnerability detection, this involves training the model to identify vulnerabilities in code snippets by leveraging the rich representations learned during pre-training.

By following these steps, PLMs can effectively transfer knowledge from the pre-training phase to the fine-tuning phase, thereby enhancing their performance on various downstream tasks, including vulnerability detection. The use of PLMs in vulnerability detection helps identify potential security issues in code by capturing subtle patterns and anomalies, thereby improving the accuracy and reliability of automated detection systems.

¹ <https://github.com/SIFT01/SIFT>.

2.2 PLMs with different architectures

Existing PLMs can be broadly categorized into three architectures: encoder-only, decoder-only, and encoder-decoder (Lu et al. 2023).

Encoder-only models: These models, such as CodeBERT (Feng et al. 2020), use only an encoder to process the input. The encoder transforms the input code into a high-dimensional representation, capturing syntactic and semantic information. Encoder-only models are particularly effective for tasks that require understanding the entire input sequence, such as code classification and code search.

Decoder-only models: These models utilize only a decoder to generate outputs from a given context, making them well-suited for tasks like code completion and code generation. An example of a decoder-only model is GPT-3 (Kalyan 2023), which has demonstrated strong performance in generating human-like text, including code.

Encoder-decoder models: These models, such as T5 (Raffel et al. 2020) and BART (Lewis et al. 2019), use both an encoder and a decoder. The encoder processes the input code to create a comprehensive representation, which the decoder then uses to generate the output. This architecture is beneficial for tasks that involve transforming one sequence into another, such as code translation and summarization.

2.3 Code representation

Code representation is a crucial research direction in many software engineering tasks, including program repair (Tian et al. 2020), code completion (Liu et al. 2020), vulnerability detection (Wang et al. 2020), and more. The effectiveness of code representation significantly impacts the performance of models in these tasks. Researchers have developed various methods to represent source code effectively, which can be broadly categorized into four types: feature-based methods, token-based methods, tree-based methods, and graph-based methods (Siow et al. 2022).

Feature-based methods extract specific attributes or metrics from the code, such as the number of lines, complexity metrics, or specific syntactic patterns (Zhang et al. 2019; Behl et al. 2014). These features are often manually engineered and require domain expertise to identify which aspects of the code are most relevant for a given task. While feature-based methods can provide useful insights, they are limited by their reliance on predefined features and may miss nuanced structural and semantic information.

Token-based methods treat code as a linear sequence of tokens, similar to how text is processed in natural language processing (NLP) (Hua et al. 2020?). Examples of this approach include models like CodeBERT, which leverages techniques from NLP to understand code. Token-based methods can capture some context and order of operations, but they often fail to represent the deeper hierarchical and relational structure inherent in source code.

Tree-based methods utilize hierarchical structures such as abstract syntax trees (ASTs) to represent the syntactic structure of the code (Alon et al. 2018,

2019). ASTs provide a more structured view of the code compared to linear sequences by showing the nested relationships between different elements of the code. This approach helps in understanding the syntactic constructs and scope within the code but might not fully capture the dynamic aspects of program execution.

Graph-based methods represent code using graph structures that capture various relationships within the code (Liu et al. 2020; Fernandes et al. 2018; Wang et al. 2020), such as control flow graphs (CFGs), which represent the order in which statements are executed; program dependence graphs (PDGs), which capture both data and control dependencies between statements; and code property graphs (CPGs), which combine ASTs, CFGs, and PDGs to provide a comprehensive view of the code's structure and semantics.

2.4 Research motivation

Existing GNN-based models, such as IVDetect (Li et al. 2021), often require extensive time to learn code structures. For instance, IVDetect takes up to 9 days to train and heavily depends on the quantity of downstream task data. Although the pre-training-fine-tuning paradigm alleviates these issues to some extent, current approaches still struggle to effectively absorb domain-specific structural knowledge relevant to downstream tasks (Li et al. 2021). Specifically, while general code structure knowledge enables models to learn common features such as function call relationships and variable assignments, these features may be insufficient to capture vulnerability-specific characteristics.

In the context of vulnerability detection, it is crucial for models to learn specific features related to vulnerabilities (Cai et al. 2024), such as potential vulnerability triggering conditions, the propagation paths of security vulnerabilities, and the likelihood of vulnerability exploitation. Without this specialized knowledge, models may fail to accurately identify vulnerabilities, limiting their effectiveness in real-world applications.

Therefore, our research motivation is to alleviate the aforementioned limitations through SIFT by integrating domain-specific structural knowledge during fine-tuning, thereby enhancing the model's ability to detect vulnerabilities.

3 Approach

In this section, we introduce the overall architecture of SIFT. As shown in Fig. 2, this architecture includes three main components: (1) **Preprocessing Phase**, which primarily removes various noises from the dataset, (2) **Graph Structure Information Generation Phase**, which aims at representing code text as rich graph structure information, and (3) **Multi-task Learning Phase**, which aims to enable the model to learn more structural information through a multi-task learning approach.

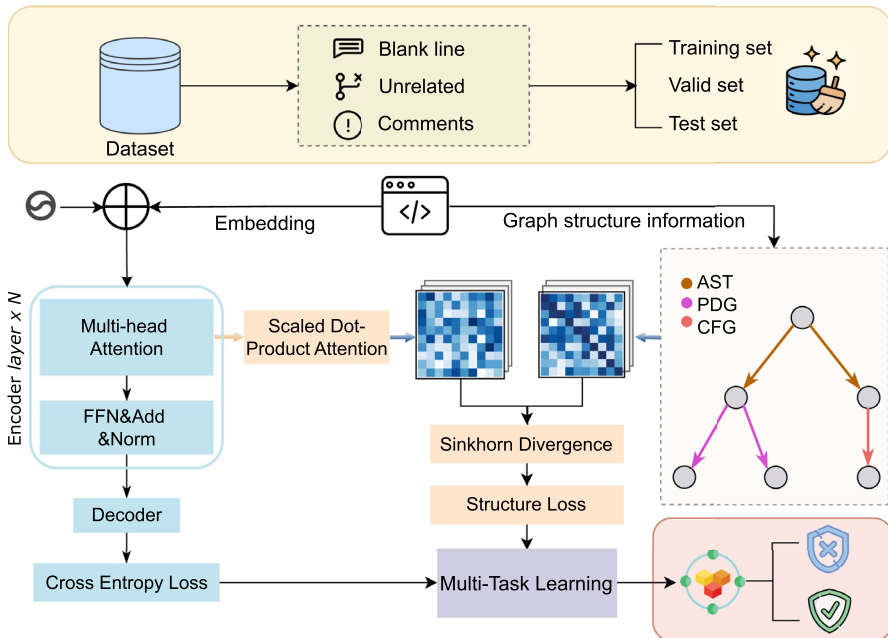


Fig. 2 The architecture of SIFT

3.1 Preprocessing phase

In the preprocessing phase, we focus on cleaning the dataset to ensure the quality and relevance of the data used for training and evaluation. This phase involves the removal of noise, which is crucial for improving the model's performance. The noise processing primarily targets three aspects: blank lines, comments, and irrelevant statements (such as debug statements).

Removal of blank lines: Blank lines in the code do not contribute any meaningful information and can introduce unnecessary complexity into the model's input (Chen et al. 2017). We systematically remove all blank lines from the dataset to streamline the code representation and reduce redundancy.

Removal of comments: While comments are useful for human understanding of the code, they do not play a role in the actual execution of the code and can distract the model from learning the structural and functional aspects of the code (Zou et al. 2019). Therefore, we strip out all inline comments, block comments, and documentation strings from the code snippets.

Removal of irrelevant statements: Irrelevant statements, such as debug prints, logging statements, and other non-functional code fragments, do not contribute to the code's logic and can introduce noise (Li et al. 2020). These statements are identified and removed to ensure that the dataset contains only the essential code relevant to the tasks of interest.

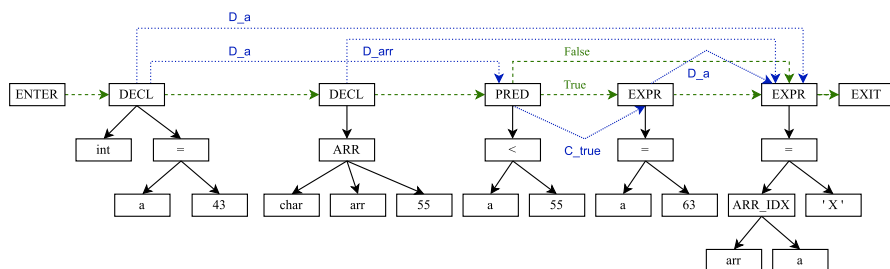
By carefully preprocessing the dataset and removing these sources of noise, we can provide the model with cleaner and more relevant data, which enhances its

Fig. 3 A snippet of C code

```

01. void foo()
02. {
03.     int a = 43;
04.     char arr[55];
05.     if (a<55)
06.     {
07.         a=63;
08.     }
09.     arr[a] = 'X'
10. }

```

**Fig. 4** An example of a code property graph (CPG)

ability to learn meaningful patterns and improves its overall performance on downstream tasks.

3.2 Graph structure information generation phase

In the graph structure information generation phase, we utilize the Joern tool² to construct a Code Property Graph (CPG) representation of the source code. The CPG integrates three critical components: the Abstract Syntax Tree (AST), the Control Flow Graph (CFG), and the Program Dependency Graph (PDG). As illustrated in Fig. 4, a C function shown in Fig. 3 is transformed into a graph consisting of various types of nodes and edges. The nodes and black edges represent the AST, capturing the syntactic structure of the function. However, the AST alone lacks the necessary semantic information to enable the model to comprehend the function's behavior fully. To address this limitation, the CPG integrates semantic information, such as data flow and control flow, into the syntactic structure provided by the AST.

Specifically, in Fig. 4, the blue edges labeled “D_a” illustrate data dependencies between nodes, indicating that the value defined in the variable “a” is utilized in the subtree of a corresponding node. Similarly, the green edges denote execution order, including conditional branches. By combining syntactic and semantic information,

² <https://joern.io/>.

the CPG provides a richer representation that enhances the model's ability to understand the function's underlying structure and behavior.

In general, the AST captures the syntactic structure of the program, the PDG models the data and control dependencies within the code, and the CFG represents all potential execution paths. By combining these graphs, the CPG provides a comprehensive representation that encompasses both structural and semantic aspects of the code. The formal definition of the CPG is presented below:

Definition 1 A CPG $G = (N, E, \lambda, V)$ is constructed from the AST, CFG, and PDG of the source code, where:

- $N = N_A$ is the set of nodes in the CPG,
- $E = E_A \cup E_C \cup E_P$ is the set of directed edges,
- $\lambda = \lambda_A \cup \lambda_C \cup \lambda_P$ is the labeling function for edges, and
- $V = V_A \cup V_E$ is the set of properties for nodes and edges.

Where, N_A represents the nodes in the AST, and E_A , E_C , and E_P are the sets of directed edges associated with the AST, CFG, and PDG, respectively. The edge labels are defined by the functions λ_A , λ_C , and λ_P . The properties of nodes and edges are described by the function $V : (N \cup E) \times K \rightarrow S$, where K is the set of property keys, and S is the set of property values. Since the CPG integrates multiple graphs, two nodes may be connected by multiple edges, representing diverse relationships within the code.

3.3 Multi-task learning phase

In the Multi-task Learning Phase, our objective is to quantify the disparity between the structural information learned by the PLMs and the code structure knowledge encapsulated in the CPG representations generated in Sect. 3.2. To achieve this, we introduce the attention matrix and the code structure matrix. Figure 2 illustrates the process of generating the attention matrix and the code structure matrix. Firstly, we generate the attention matrix. The raw code is input as a sub-token sequence into the multi-head self-attention blocks of PLM. Within these blocks, scaled dot-product attention (Vaswani et al. 2017) is computed between sub-tokens. These attention values are then extracted and used to construct the attention matrix. Next, we generate the code structure matrix. The raw code is first parsed into a CPG representation. Next, we treat CPG as an undirected graph, where the distances between tokens correspond to the shortest path lengths between the corresponding nodes in CPG. We choose to use an undirected graph instead of a directed one to simplify the computation of shortest paths and ensure the efficient generation of the structure matrix. In a directed graph, the presence of unreachable nodes would lead to more infinite distances in the structure matrix, resulting in increased sparsity. This sparsity could negatively impact the effectiveness of the subsequent learning process, as the model may struggle to capture meaningful structural relationships. Based on

these distances, we construct the code structure matrix, representing the structural relationships within the code.

Notice the shapes of the attention matrix and the distance matrix are different. To facilitate a comparison between the code structure matrix and the attention matrix, we convert the code structure matrix from token-level to sub-token-level, ensuring both matrices share the same shape.

We present an example using the code snippet:

```
num_lens = get_bits(gb, 5)
```

At the token level, the input code is split into syntactically meaningful units, such as variables, operators, and function calls. The resulted tokens are as follows:

Tokens: ['num_lens' '=' 'get_bits' '(' 'gb' ',' '5' ')']

However, PLMs, such as BERT, operate at the sub-token level using tokenization methods like WordPiece or Byte-Pair Encoding (BPE). Longer or more complex tokens are further broken down into smaller sub-units to enhance semantic understanding. For the example above, the sub-tokenized sequence becomes:

Sub-Tokens: ['num' '_' 'lens' '=' 'get' '_' 'bits' '(' 'gb' ',' '5' ')']

To align the code structure matrix (originally defined at the token level) with the attention matrix (which operates at the sub-token level), we map token-level relationships onto their corresponding sub-tokens. This is achieved through the following steps: ① Each token's distance values are propagated uniformly to all its sub-tokens. ② Sub-tokens derived from the same token are assigned a distance of zero between each other, ensuring consistency.

For example, given a token-level distance matrix:

	num_lens = get_bits (			
num_lens	0	1	2	3
=	1	0	1	2
get_bits	2	1	0	1
(	3	2	1	0

The corresponding sub-token-level distance matrix is:

	num _ lens = get _ bits						
num	0	0	0	1	2	2	2
_	0	0	0	1	2	2	2
lens	0	0	0	1	2	2	2
=	1	1	1	0	1	1	1
get	2	2	2	1	0	0	0
_	2	2	2	1	0	0	0
bits	2	2	2	1	0	0	0

Where, the sub-tokens num, _, and lens inherit the same distances as their parent token num_lens, while maintaining zero distance among themselves. By converting

the code structure matrix from token-level to sub-token-level, we bridge the gap between graph-based representations and the attention mechanism of pre-trained models. This alignment not only ensures compatibility but also leverages the finer-grained insights provided by sub-tokenization, enabling a more comprehensive understanding of the code structure.

Next, following the approach of Weng (2019), we use the Wasserstein distance to quantify the discrepancy between the attention matrix and the code structure matrix. The Wasserstein distance is a smooth metric for measuring the distance between two probability distributions, even when these distributions reside on non-overlapping low-dimensional manifolds. However, computing the Wasserstein distance is computationally expensive (Chizat et al. 2020). To mitigate the computational cost, we leverage Geomloss (Lacombe et al. 2023) to compute the Sinkhorn divergence as a substitute. The Sinkhorn divergence is a positive definite approximation of the Wasserstein distance that achieves similar results with significantly lower computational cost (Chizat et al. 2020).

Since the values of the scaled dot-product attention and the structure matrix are not on the same scale, the Sinkhorn divergence is not suitable for measuring the difference between matrices with values of different magnitudes. To address this issue, we introduce a linear layer that applies scaling and bias to the code structure matrix. This linear layer, referred to as the structural encoder, is treated as a learnable parameter integrated into the model. We treat the attention matrix and the code structure matrix as two distributions. By applying the structural encoder to scale and bias the code structure matrix, we use the Sinkhorn divergence to measure the discrepancy between the two matrices to obtain the structural knowledge loss. The specific details are as follows:

$$\mathcal{L}_i = \text{Sinkhorn divergences}(\mathbf{X}, \text{Linear}(\mathbf{Y})) \quad (1)$$

$$\mathcal{L}_{\text{structural knowledge}} = \frac{\sum_{i=1}^h \mathcal{L}_i}{h} \quad (2)$$

where X represents the attention matrix, Y represents the code structure matrix, and h denotes the number of attention heads. Next, we use a multi-task fine-tuning approach (Lamsiyah et al. 2023) to jointly optimize the objective function of the downstream task and the structural knowledge loss. This method enables the model to learn the structural information of the code during the backpropagation stage. The final objective function is as follows:

$$\mathcal{L}_{\text{final}} = (1 - \omega) \cdot \mathcal{L}_{\text{vulnerability}} + \omega \cdot \mathcal{L}_{\text{structural knowledge}} \quad (3)$$

where $\mathcal{L}_{\text{vulnerability}}$ represents the objective function of the downstream task, which is the cross-entropy loss in vulnerability detection. ω is a hyperparameter that controls the weight of the structural knowledge loss. We will discuss the sensitivity of SIFT to the hyperparameter ω in Sect. 6, providing insights to guide practitioners in fine-tuning the model.

Finally, after training the model, we evaluate it on the test set. Our experimental results are presented in Sect. 5.

4 Experimental setup

4.1 Research questions

To evaluate the effectiveness of our proposed SIFT, we want to answer the following four research questions (RQs):

RQ1: How effective is SIFT in vulnerability detection?

Motivation: We designed this RQ to verify the effectiveness of our proposed approach SIFT, in the field of vulnerability detection. To evaluate its performance, we will compare SIFT against nine SOTA vulnerability detection approaches. This comparison aims to demonstrate whether SIFT offers significant improvements in identifying security vulnerabilities in code, thereby validating the benefits of incorporating structural knowledge during the fine-tuning process.

RQ2: How effective is using CPG as structural information?

Motivation: We designed this RQ to validate the effectiveness of using CPG as a representation, which integrates AST, PDG, and CFG into a unified data structure. To determine the effectiveness of this structural information, we will conduct an ablation study to evaluate the impact of each component individually. This analysis will help to understand the contribution of each structural element and verify whether their integration improves the performance of the vulnerability detection task.

RQ3: How do different PLMs impact the performance of SIFT?

Motivation: We designed this RQ to verify how different PLM architectures impact the performance of SIFT. We plan to experiment with three distinct PLM architectures to determine whether SIFT is effective across various architectures. This will help us understand the generalizability and robustness of SIFT in leveraging different underlying PLMs for vulnerability detection tasks.

RQ4: How does SIFT perform in scenarios with scarce data?

Motivation: We designed this RQ to verify the performance of SIFT in scenarios with scarce data. By evaluating SIFT's effectiveness in data-constrained environments, we aim to determine its practicality and robustness when the amount of available training data is limited. This assessment will help us understand SIFT's capability to generalize from limited data, which is crucial for real-world applications where extensive labeled datasets may not always be available.

4.2 Datasets

In our study, we used three widely applied datasets: FFmpeg+Qemu (Zhou et al. 2019), Chrome+Debian (Chakraborty et al. 2022), and Big-Vul (Fan et al. 2020). These datasets have been used in numerous studies, such as GRACE (Lu et al. 2024)

and AMPLE (Wen et al. 2023). The detailed information about the datasets is as follows.

- **FFmpeg+Qemu** (Zhou et al. 2019). The FFMpeg+Qemu dataset, also known as Devign, was manually labeled and derived from two open-source C projects: FFMpeg and Qemu. It includes 10,067 functions marked as vulnerable and 12,294 functions marked as non-vulnerable.
- **Chrome+Debian** (Chakraborty et al. 2022). The Chrome+Debian dataset, also known as Reveal, was sourced from the Linux Debian Kernel and Chromium projects. It contains 1664 vulnerable functions and 16,505 non-vulnerable functions.
- **Big-Vul** (Fan et al. 2020). The Big-Vul dataset was collected from over 300 open-source C/C++ projects on GitHub and covers 91 distinct vulnerability types listed in the Common Vulnerabilities and Exposures (CVE) database from 2002 to 2019. It includes 10,547 functions identified as vulnerable and 179,299 functions identified as non-vulnerable.

Table 1 lists the detailed information of these three datasets, including the total number of samples, the number of vulnerable samples, the number of non-vulnerable samples, and the ratio of vulnerable samples.

4.3 Baseline methods

To evaluate the effectiveness of SIFT, we compare it with three token-based methods (i.e., VulDeePecker (Li et al. 2018), SySeVR (Li et al. 2022), and (Russell et al. 2018)), three graph-based methods (i.e., Devign (Zhou et al. 2019), Reveal (Chakraborty et al. 2022), and AMPLE (Wen et al. 2023)) and three LLM-based methods (i.e., LineVul (Fu and Tantithamthavorn 2022), CodeBERT (Feng et al. 2020), and GRACE (Lu et al. 2024)). These baselines are widely recognized and highly regarded in the field of vulnerability analysis. Another advantage is that these methods are open-source. We conducted a simple statistical analysis, and as of July 1, 2024, these baselines have been cited as follows: 1001, 653, 587, 804, 441, 25, 165, 2190, and 5 times, respectively. Moreover, their performance also ranks among the top in the field of vulnerability detection. We show the details of these nine baselines as follows.

Table 1 Statistics of the dataset used in our vulnerability detection study

Dataset	Samples	Vul	Non-vul	Vul ratio (%)
FFmpeg+Qemu (Zhou et al. 2019)	22,361	10,067	12,294	45.02
Chrome+Debian (Chakraborty et al. 2022)	18,169	1664	16,505	9.16
Big-Vul (Fan et al. 2020)	179,299	10,547	168,752	5.88

- **VulDeePecker** (Li et al. 2018). VulDeePecker divides code into code gadgets and uses a bidirectional LSTM network to extract features and perform binary classification.
- **SySeVR** (Li et al. 2022). SySeVR extracts code representations that incorporate both syntactic and semantic code features. It then utilizes a bidirectional gated recurrent unit (BGRU) network to perform vulnerability detection.
- **Russell et al.** (Russell et al. 2018). Russell et al. first utilize static analysis tools to label the dataset, then employ convolutional neural networks and other techniques to detect vulnerabilities.
- **Devign** (Zhou et al. 2019). Devign builds code structure representations by extracting graph information from functions, then feeds this graph information into a Gated Graph Neural Network (GGNN) to categorize the code.
- **Reveal** (Chakraborty et al. 2022). Reveal constructed multiple directed graphs from source code and then utilized a GGNN to analyze these graphical representations for vulnerability detection.
- **AMPLE** (Wen et al. 2023).AMPLE simplifies and enhances code structure graphs, using Graph Convolutional Networks (GCN) to obtain graph embeddings. It then utilizes a Multi-Layer Perceptron (MLP) for classification and detection.
- **LineVul** (Fu and Tantithamthavorn 2022). LineVul proposes a vulnerability prediction method based on Transformers. It trains an additional sequence classification model on top of a pre-trained LLM to predict which functions and code lines contain vulnerabilities.
- **CodeBERT** (Feng et al. 2020). CodeBERT is a pre-trained model for programming languages, leveraging the BERT architecture to create a language model specifically tailored for code. It learns from both source code and natural language, allowing it to understand and generate code with high accuracy.
- **GRACE** (Lu et al. 2024). GRACE introduces a novel approach by transforming code into graph-structured information to be fed into a large language model (LLM). Additionally, it designs a new retrieval method to obtain few-shot examples for contextual learning, thereby enhancing software vulnerability detection based on LLMs.

We reproduce the results of these baselines by replicating their open-source code. The exception is Devign (Zhou et al. 2019), as the original Devign is not open-sourced, so we use Chakraborty et al. (2022) proposed third-party open-source code to reproduce it.

4.4 Performance metrics

To evaluate the performance of SIFT, we use the following metrics: accuracy, precision, recall, and F1 score. These metrics are widely recognized in the software engineering community and have been utilized in numerous studies on vulnerability detection, such as LineVul (Fu and Tantithamthavorn 2022), ReGVD (Nguyen et al. 2022), PLBART (Ahmad et al. 2021), and the baseline models mentioned in

the article. These metrics are defined based on true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN). True Positive (TP): The model correctly identifies a code snippet as vulnerable. True Negative (TN): The model correctly identifies a code snippet as not vulnerable. False Positive (FP): The model incorrectly identifies a code snippet as vulnerable when it is not. False Negative (FN): The model incorrectly identifies a code snippet as not vulnerable when it actually is.

Using these values, we define our performance metrics as follows:

Accuracy. Measures the overall correctness of the model and is defined as the ratio of correctly predicted instances (both TP and TN) to the total number of instances. The formula is defined as follows:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (4)$$

Precision. Indicates the proportion of positive identifications that were actually correct. It focuses on the quality of the positive predictions. The formula is defined as follows:

$$Precision = \frac{TP}{TP + FP} \quad (5)$$

Recall. *Recall* measures the ability of a model to detect all actual samples containing vulnerabilities, i.e., the proportion of actual samples containing vulnerabilities that are correctly predicted as positive by the model. The formula is defined as follows:

$$Recall = \frac{TP}{TP + FN} \quad (6)$$

F1 Score. *F1 score* represents a balance between precision and recall and is calculated as the harmonic mean of the two. The formula is defined as follows:

$$F1\ score = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (7)$$

These metrics provide a comprehensive evaluation of the model's performance, helping us understand its strengths and weaknesses in detecting vulnerabilities.

4.5 Implementation details

We divided the dataset into training, testing, and validation sets with a ratio of 8:1:1, following the typical dataset split configuration used in previous studies (Lu et al. 2024; Wen et al. 2023; Li et al. 2022). Notably, to ensure the fairness of our experiments, we used the same testing set for each baseline as well as for our method. For the reproduction of Devign (Zhou et al. 2019), we replicated the experiments using the methods provided by ReVeal (Chakraborty et al. 2022), as the original code was not available. All our experiments were conducted on an NVIDIA A100-SXM4-80GB server, running Ubuntu 20.04 and CUDA 11.8.

5 Experimental results

5.1 RQ1: How effective is SIFT in vulnerability detection?

Approach. To answer RQ1, we use the four performance metrics introduced in Sect. 4: accuracy, precision, recall, and F1 score. We compare SIFT against nine SOTA baseline methods for vulnerability detection. The experiments are conducted on three commonly used vulnerability datasets, which have been detailed in Sect. 4. To ensure fairness in our comparisons, we maintain consistent training and testing datasets across all methods.

Result. Table 2 presents the overall results of all baseline models and SIFT across four evaluation metrics. Overall, SIFT achieved superior results on all three datasets, outperforming all nine mentioned token-based, graph-based, and LLM-based methods. Specifically, out of the 12 results across the three datasets and four performance metrics, SIFT showed the best performance in 9 cases and ranked within the top 3 in the remaining 3 cases. Notably, in terms of the F1 score, SIFT achieved the best performance, with absolute improvements of 1.74%, 10.19%, and 2.87% over the current best baseline on the three datasets, respectively. Then, to check whether the performance difference between SIFT and baselines is significant, we conduct Wilcoxon signed-rank tests (Wilcoxon 1992) at a confidence level of 95%. The p value is less than 0.05, indicating that the performance improvement of SIFT compared to the baselines is statistically significant.

The experimental results also indicate that among different types of baselines, LLM-based methods such as LineVul, CodeBERT, and GRACE demonstrated the best performance, followed by the three graph-based methods Devign, ReVeal, and AMPLE. The token-based methods showed the least advantage. For instance, in the FFMpeg+Qemu dataset, the gray cells, which denote the top 3 best results, appeared 9 times in LLM-based methods, 3 times in graph-based methods, and 0 times in token-based methods. This disparity might be attributed to the fact that token-based methods are more adept at capturing the sequential information of the code but tend to overlook the structural information. In contrast, graph-based methods are better at capturing the structural information of the code, which is beneficial for vulnerability detection. Moreover, LLM-based methods provide a

Table 2 n SIFT and the baselines on the three datasets in vulnerability detection

Metrics(%)	Dataset	FFmpeg+Qemu [12]				Chrome+Debian [13]				Big-Vul [14]			
		Accuracy	Precision	Recall	F1 score	Accuracy	Precision	Recall	F1 score	Accuracy	Precision	Recall	F1 score
Token-based	Vuldeepecker	49.91	46.05	32.55	38.14	76.37	21.13	13.10	16.17	81.19	38.44	12.75	19.15
	syssevr	47.85	46.06	58.81	51.66	74.33	40.07	24.94	30.74	90.10	30.91	14.08	19.34
	Russell et al.	57.60	54.76	40.72	46.71	68.51	16.21	52.68	24.79	86.85	14.86	26.97	19.17
Graph-based	Devign	56.89	52.50	64.67	57.95	87.49	31.55	36.65	33.91	92.78	30.61	15.96	20.98
	Reveal	61.07	55.50	69.21	61.60	81.77	31.55	61.14	41.62	87.14	17.22	34.04	22.87
	AMPLE	63.01	53.26	83.21	64.95	90.72	50.06	43.28	46.42	93.14	29.98	34.58	32.11
LLM-based	Linevul	62.75	63.98	50.51	56.45	92.43	48.86	41.35	44.79	95.82	90.68	83.31	86.84
	CodeBERT	62.08	62.94	58.70	60.74	88.21	48.95	35.35	41.06	94.24	47.90	31.99	38.36
	GRACE	59.78	53.94	82.13	65.11	89.73	33.21	61.53	43.13	90.73	32.52	39.08	35.50
Ours	SIFT	65.27	64.46	68.13	66.24	93.84	49.35	52.88	51.15	97.16	91.97	86.83	89.33

The best result for each metric is highlighted in bold. The cells shaded in grey represent the performance of the top 3 best approaches for each metric, with darker shading indicating better performance

more comprehensive solution by leveraging extensive code-related knowledge acquired through pre-training on large-scale code datasets.

Summary for RQ1: SIFT outperforms the nine baseline methods in terms of F1 score. Specifically, on three datasets, SIFT achieves relative improvements of 1.74%, 10.19%, and 2.87% over the current best-performing baseline, respectively.

5.2 RQ2: How effective is using CPG as structural information?

Approach. To answer RQ2, we conducted an ablation study. In this study, we respectively removed AST, PDG, and CFG components and compared the performance of these variants. By isolating each component, we aim to evaluate the contribution of each type of structural information and demonstrate the effectiveness of using CPG as a representation.

Result. The results of different variants are presented in Table 3. We found that all variants performed lower than CPG, indicating the contribution of all components to the overall performance of CPG. Specifically, without the AST, the average accuracy, precision, recall, and F1 score decreased by 2.45%, 2.06%, 2.16%, and 2.19%, respectively, across the three datasets. Without the CFG component, these metrics decreased by 2.77%, 2.24%, 1.88%, and 2.12%, respectively. While both AST and CFG provide critical insights, their roles are complementary. The AST focuses on structural and semantic correctness, making it valuable for detecting vulnerabilities at the syntactic level. On the other hand, CFG extends this understanding by providing execution-level insights, making it indispensable for addressing logic-driven issues. The marginally greater impact of CFG in certain metrics, such as precision and accuracy, indicates that control flow information contributes more directly to reducing false positives and refining model predictions.

Table 3 Results of ablation study for different components of graph structural information

Dataset	Method	Accuracy	Precision	Recall	F1 score
FFMPeg+Qemu	w/o AST	63.55	63.10	66.71	64.85
	w/o PDG	61.28	59.37	66.69	62.82
	w/o CFG	62.83	62.47	67.23	64.76
	GPG	65.27	64.46	68.13	66.24
Chrome+Debian	w/o AST	90.98	47.95	51.63	49.72
	w/o PDG	89.05	47.52	51.22	49.30
	w/o CFG	92.38	48.60	51.40	49.96
	GPG	93.84	49.35	52.88	51.15
Big-Vul	w/o AST	95.56	90.83	85.05	87.84
	w/o PDG	93.51	89.70	84.17	86.85
	w/o CFG	94.23	90.03	85.52	87.72
	GPG	97.16	91.97	86.83	89.33

The bold font indicates better performance

Notably, among these components, the PDG contributed the most to the overall performance, improving the F1 score by 5.44%, 3.75%, and 2.86% on the three datasets, respectively. This is likely because PDG captures both data dependencies and control dependencies within the program, providing a comprehensive view of how variables and operations interact throughout the code. Unlike AST, which primarily focuses on the syntactic structure, and CFG, which emphasizes the control flow, PDG integrates both dynamic and static aspects of the program, making it particularly effective for identifying complex vulnerability patterns.

The comparison among the variants also reveals that while all components contribute to the performance of CPG, their importance varies depending on the nature of the datasets and the types of vulnerabilities being analyzed. For instance, PDG's stronger impact aligns with its ability to model intricate relationships between program variables and execution flow, which are critical for detecting data-flow-related vulnerabilities like resource leaks or uninitialized variables. On the other hand, AST and CFG components, while individually less impactful, provide complementary benefits by enhancing syntactic understanding and control flow reasoning, respectively.

Summary for RQ2: Our experimental results indicate that the components within CPG effectively enhance SIFT's performance, with the PDG contributing the most to the overall performance improvement.

5.3 RQ3: How do different PLMs impact the performance of SIFT?

Approach. To answer RQ3, we conducted experiments using three different architectures of PLMs: the encoder-only CodeBERT (Feng et al. 2020), the decoder-only CodeGPT (Lu et al. 2021), and the encoder-decoder CodeT5 (Wang et al. 2021). We then compared the performance of SIFT across these different architectures. This comparison aims to assess how the underlying architecture of PLMs affects the performance of SIFT in vulnerability detection tasks.

Table 4 Results using different pre-trained language models on three datasets

Dataset	Method	Accuracy	Precision	Recall	F1 score
FFMPeg+Qemu	CodeBERT	63.61	64.12	61.53	63.82
	CodeGPT	63.84	62.71	67.36	64.95
	CodeT5	65.27	64.46	68.13	66.24
Chrome+Debian	CodeBERT	91.04	49.87	47.11	48.45
	CodeGPT	89.73	48.86	49.59	49.22
	CodeT5	93.84	49.35	52.88	51.15
Big-Vul	CodeBERT	95.71	71.33	66.84	69.01
	CodeGPT	94.85	60.59	72.45	65.99
	CodeT5	97.16	91.97	86.83	89.33

The bold font indicates better performance

Result. Table 4 presents the results of SIFT using different PLMs across three datasets: FFMpeg+Qemu, Chrome+Debian, and Big-Vul. From the results, we can see that the encoder-decoder model CodeT5 outperformed both the encoder-only model CodeBERT and the decoder-only model CodeGPT across most evaluation metrics and datasets. Specifically, CodeT5 achieved the highest F1 scores, with improvements of up to 3.92% over the best-performing baselines on the Chrome+Debian dataset and up to 29.45% on the Big-Vul dataset. The superior performance of CodeT5 can be attributed to its encoder-decoder architecture, which effectively captures both the contextual information and the sequential dependencies within the code. This comprehensive representation allows for more accurate vulnerability detection. These findings highlight the importance of choosing an appropriate PLM architecture for enhancing the effectiveness of SIFT in vulnerability detection tasks.

Summary for RQ3: The choice of PLMs significantly impacts SIFT’s performance. Experimental results indicate that the encoder-decoder CodeT5 achieves the best performance.

5.4 RQ4: How does SIFT perform in scenarios with scarce data?

Approach. To answer RQ4, we conducted experiments by fine-tuning SIFT on datasets of varying sizes. Specifically, we randomly selected subsets of the training data to create datasets containing 20%, 40%, 60%, and 80% of the original data. This approach enables us to evaluate how the performance of SIFT changes with different amounts of available fine-tuning data, simulating scenarios with scarce data.

Table 5 Results from different training set resource scenarios

Dataset	Method	Accuracy	Precision	Recall	F1 score
FFMPeg+Qemu	20%	49.61	50.12	50.34	50.23
	40%	55.34	58.01	57.40	57.70
	60%	59.15	61.54	60.95	61.24
	80%	63.91	63.07	67.12	65.03
	100%	65.27	64.46	68.13	66.24
Chrome+Debian	20%	74.19	40.37	40.83	40.60
	40%	80.12	45.62	47.50	46.54
	60%	85.24	47.94	50.54	49.21
	80%	89.85	49.13	52.49	50.75
	100%	93.84	49.35	52.88	51.15
Big-Vul	20%	86.11	60.31	71.49	65.43
	40%	87.59	72.66	75.74	74.17
	60%	93.04	84.64	80.82	82.69
	80%	95.77	90.92	84.04	87.34
	100%	97.16	91.97	86.83	89.69

The bold font indicates better performance

Result. The results of SIFT’s performance with different amounts of training data are presented in the Table 5. We observed that the performance of SIFT improved consistently with the increase in the amount of training data across all three datasets: FFMpeg+Qemu, Chrome+Debian, and Big-Vul.

The results demonstrate that SIFT performs well even with smaller subsets of data, though its performance improves as more training data becomes available. Specifically, for the FFMpeg+Qemu dataset, the F1 score increased from 50.23% with 20% of the data to 66.24% with 100% of the data, showing a consistent improvement. Similar trends were observed for the Chrome+Debian and Big-Vul datasets. An additional interesting finding is that on the Chrome+Debian dataset, using only 60% of the data, SIFT achieves an F1 score of 49.21%, which is higher than all nine baselines evaluated in RQ1. This highlights the effectiveness of our approach.

These findings suggest that SIFT can still achieve considerable performance with limited data, making it a robust choice for vulnerability detection in scenarios where data is scarce. However, to fully leverage its capabilities and achieve the best performance, providing a larger amount of training data is beneficial.

Summary for RQ4: The performance of SIFT improves with increasing training data volume, reaching its peak when the data amount is at 100%. Additionally, the most significant increase in F1 score occurs when the data volume grows from 20% to 40%.

6 Discussion

6.1 How effectively does SIFT truly learn structural information?

To further analyze whether SIFT truly learns structural knowledge, we compared it with existing SOTA fine-tuning methods: fine-tuning and prompt tuning. Prompt tuning aims to convert the downstream task objectives of fine-tuning into pre-training tasks (Liu et al. 2022), addressing the inherent gap between the pre-training task and the vulnerability detection task. Here, we used a prompt template: “The code

Table 6 Performance comparison of different fine-tuning methods on three datasets

Dataset	Method	Accuracy	Precision	Recall	F1 score
FFMPeg+Qemu	fine-tuning	63.56	58.55	54.97	56.70
	prompt tuning	63.97	63.48	56.17	59.60
	SIFT	65.27	64.46	68.13	66.24
Chrome+Debian	fine-tuning	85.92	41.13	36.19	38.50
	prompt tuning	87.11	43.75	36.46	39.77
	SIFT	93.84	49.35	52.88	51.15
Big-Vul	fine-tuning	96.01	50.94	84.65	63.60
	prompt tuning	98.20	93.39	76.45	84.07
	SIFT	97.16	91.97	86.83	89.33

The bold font indicates better performance

[X] is [Z].” The verbalizers used were “non-vulnerable” and “vulnerable”. This prompt template and verbalizer design have been proven to provide higher performance (Lu et al. 2023).

We compared SIFT with these two methods that do not enhance structural knowledge. To ensure the fairness of the experiments, we used the same model, CodeT5, for all methods. The results are shown in the Table 6. Out of the 12 experiments across three datasets, SIFT achieved the best performance in 10 of them. Specifically, the F1 scores on the three datasets were higher by 13.98%, 30.74%, and 23.36%, respectively.

These results demonstrate that SIFT effectively learns and utilizes structural information, outperforming both fine-tuning and prompt tuning in the majority of cases. This highlights the importance of considering structural information in vulnerability detection tasks and suggests that methods like SIFT, which incorporate such information, are more effective.

6.2 How do different hyperparameter settings affect the performance of SIFT?

In the multi-task learning phase, we propose a combination of structural loss and cross-entropy loss to enhance vulnerability detection capabilities. To better understand the impact of the weight of structural loss in SIFT, we conducted experiments with different hyperparameter settings.

We set ω to values ranging from 0 to 1 and conducted experiments on the CodeT5 model. The evaluation metrics for the three datasets are presented in Fig. 5. As ω increases from 0 to 0.4 or 0.5, the F1 score improves steadily, indicating that incorporating structural knowledge helps capture critical code features and enhances the model’s ability to detect vulnerabilities. The peak performance is observed at $\omega = 0.4$ for FFmpeg+Qemu and Chrome+Debian, with F1 scores of 66.24% and 51.15%, respectively, and at $\omega = 0.5$ for BigVul, with an F1 score of 89.33%.

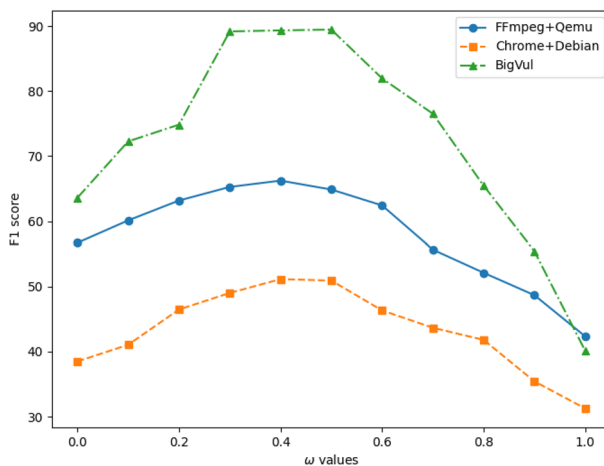


Fig. 5 The influence of different values of ω on F1 score

Beyond this point, as ω increases further, the F1 score starts to decline. For example, at $\omega = 0.7$, the F1 scores decrease to 55.67%, 43.67%, and 76.52%, highlighting the diminishing returns of structural loss when it outweighs the primary task loss. This suggests that overemphasizing structural knowledge disrupts the balance between learning code structures and focusing on the vulnerability detection task.

The results also reveal differences in the sensitivity of datasets to the structural loss weight. BigVul, being the largest and most diverse dataset, exhibits a significant performance boost as ω increases, reaching a high F1 score of 89.33%. In contrast, Chrome+Debian, which contains less diverse vulnerabilities, shows a relatively moderate improvement, peaking at 51.15%. This disparity indicates that structural knowledge is particularly beneficial for datasets with complex and varied code patterns.

6.3 Threats to validity

In this subsection, we primarily outline the potential threats to the validity of our research.

Internal threat. The quality of the datasets used might introduce internal threats. If the datasets contain noisy, incomplete, or unrepresentative samples, this could affect the performance and evaluation of SIFT. To mitigate this threat, we selected the most widely used benchmark datasets in the vulnerability detection field: FFmpeg+Qemu (Zhou et al. 2019), Chrome+Debian (Chakraborty et al. 2022), and Big-Vul (Fan et al. 2020). As of July 1, 2024, their Google Scholar citations have reached 804, 441, and 268 times respectively, indicating their broad acceptance and use in the research community. It is worth noting, as Sejfia et al. (2024) pointed out, that in the Chrome+Debian dataset, 22% of the vulnerabilities are categorized as MBUs (Multiple Base Units), i.e., vulnerabilities that span multiple base units. For such vulnerabilities, correct detection requires all comprising base units to be identified as vulnerable. In contrast to their focus on MBUs, our approach is designed to work at the function level, where each function is labeled as “vulnerable” or “non-vulnerable.” While this means we do not explicitly model MBUs, our methodology leverages advanced representations of code (e.g., Code Property Graphs, Abstract Syntax Trees, and Control Flow Graphs) to capture patterns that can generalize to both single-unit and multi-unit vulnerabilities. This distinction ensures that our method is robust across different types of vulnerability patterns, even when working within the function-level constraints of the selected datasets.

External threat. To reproduce the baseline, we meticulously followed the methods described in the original papers and used the corresponding open-source code. However, the unavailability of implementation details and hyperparameters for Devign (Zhou et al. 2019). To mitigate this threat, our replication was guided by the implementation of Reveal (Chakraborty et al. 2022).

Construct threat. The construct threats arise from the representativeness of the baselines used in our vulnerability detection experiments. When analyzing the impact of different architecture models on SIFT’s performance in RQ3, we were limited by computational resources and could not include more or larger PLMs.

However, our primary focus was on studying the effect of model architectures on performance. To mitigate this threat, we selected three SOTA PLMs that represent different architectures: encoder-only (CodeBERT (Feng et al. 2020)), decoder-only (CodeGPT (Lu et al. 2021)), and encoder-decoder (CodeT5 (Wang et al. 2021)). This choice ensures that our analysis covers a broad range of architectural designs, providing a comprehensive evaluation of how different model structures affect the performance of SIFT.

Additionally, another potential construct threat arises from our decision to treat the code property graph (CPG) as an undirected graph during the Multi-task Learning phase. While directed edges in the original CPG encode precise information about control flow and data flow directionality, we opted for an undirected representation to simplify the modeling process and focus on capturing structural relationships and shortest-path dependencies. This design choice not only reduces the complexity of graph traversal and computation but also prevents the structure matrix from becoming overly sparse. In a directed graph, the presence of unreachable nodes could result in numerous infinite distances in the structure matrix, increasing sparsity and potentially reducing the effectiveness of the learning process. By using an undirected graph, we ensure better connectivity within the graph, facilitating more robust structural relationship modeling.

Importantly, many vulnerabilities, such as resource leaks or improper variable usage, depend more on the overall structural relationships between nodes than on the specific edge directions. However, we acknowledge that this simplification may lose some directional information, which could affect cases where edge direction is critical for vulnerability detection (e.g., control flow loops or data dependency chains). Future work can explore the impact of incorporating directed graphs to capture fine-grained control and data flow semantics more comprehensively.

Conclusion threat. In our study, we tested only on C/C++ source code. However, in principle, our approach should also be effective for a broader range of programming languages, since PLMs are exposed to multiple languages during pre-training, such as Java and Python (Hou et al. 2023). This requires further investigation and represents a promising direction for future research.

7 Related work

In this section, we examine two related areas of work: vulnerability detection based on deep learning and tuning approaches for pre-trained language models. Subsequently, we underscore the pioneering aspects of our work.

7.1 Vulnerability detection based on deep learning

In the early research of software vulnerabilities, researchers typically utilized software metrics collected manually by security experts as features (Pham et al. 2010). These metrics encompass a spectrum of attributes, including code complexity, software structure, and historical vulnerability data. However, such approaches often

encounter limitations in scalability and efficacy due to the inherent subjectivity and labor-intensive nature of manual data collection and feature extraction (Gonzalez et al. 2019).

The advent of deep learning methods has revolutionized the landscape of vulnerability detection by offering automated and data-driven techniques for feature extraction and model training (Lu et al. 2024; Cao et al. 2024). For instance, Devign (Zhou et al. 2019) was proposed to learn various vulnerability features through code property graphs (CPGs) and graph neural networks (GNNs). IVDetect (Li et al. 2021) leverages graph convolutional networks (GCN) and context vectors to predict vulnerabilities. Similarly, AMPLE (Wen et al. 2023) employs an Edge-aware Graph Convolutional Network (EA-GCN) for vulnerability detection. These methods harness the power of graph-based representations to extract intricate patterns and relationships from source code, thereby enhancing the effectiveness and efficiency of vulnerability detection systems.

Recently, a series of pre-trained code models have been proposed, demonstrating promising performance in vulnerability detection tasks. These methods leverage knowledge obtained from extensive datasets to initialize the models and then fine-tune them based on data specific to vulnerability detection. This approach effectively harnesses the power of transfer learning, enabling pre-existing models to adapt to the task of vulnerability detection. For example, VulDeBERT (Kim et al. 2022) fine-tunes the BERT model for vulnerability detection on vulnerable code datasets. Fu and Tantithamthavorn (2022) proposed LineVul, a Transformer-based approach for predicting vulnerabilities at the line level. Lu et al. (2023) proposed prompt tuning for vulnerability detection, achieving superior performance compared to fine-tuning on three vulnerability datasets. Yusuf and Jiang (2024) presented instruction tuning for software vulnerability detection.

Unfortunately, while these methods have shown effectiveness in certain scenarios, they struggle to incorporate structural knowledge during fine-tuning, hindering their ability to fully capture nuanced patterns inherent in software vulnerabilities (Colon-Hernandez et al. 2021).

7.2 Tuning approaches for pre-trained language models

Pre-trained language models (PLMs) have revolutionized software engineering tasks by providing powerful representations learned from large-scale corpora (Li et al. 2023). However, to effectively apply these models to specific downstream tasks, fine-tuning is essential. Fine-tuning allows PLMs to adapt their generalized knowledge to the nuances and specific requirements of target tasks, thereby enhancing their performance and applicability. Fine-tuning approaches can be broadly categorized into weight tuning and input engineering. In this section, we explore various fine-tuning approaches within these categories, focusing on their applications in software engineering.

7.2.1 Weight tuning

Weight tuning involves modifying the model's internal parameters to better suit a particular task. Two common methods are standard full fine-tuning and adapter tuning (Zhang et al. 2023).

Standard full fine-tuning updates all or a substantial subset of a PLM's parameters using a task-specific dataset. This method aims to minimize the loss on the target task, thereby capturing its specific features. Feng et al. (2020) demonstrated that fine-tuning CodeBERT on specific code-related tasks can significantly improve performance across various code understanding and generation tasks, such as code search and code-to-documentation generation. Guo et al. (2020) showed that GraphCodeBERT merged data flow information during pre-training and achieved excellent performance in tasks that require a deeper understanding of code semantics through fine-tuning.

Adapter tuning introduces small, task-specific layers (adapters) into the PLMs, keeping most of the original model's parameters frozen. This method is more efficient and facilitates easier transfer across tasks. Ayupov and Chirkova (2022) tested adapter tuning and LoRA methods on four code intelligence tasks, including code summarization, code generation, code translation, and code clone detection. They found that while adapter tuning methods can achieve performance comparable to or even better than full fine-tuning on code understanding tasks, their performance on code generation tasks is inferior to full fine-tuning. Wang et al. (2023) achieved good results using adapter tuning on code search and code summarization tasks, demonstrating its effectiveness in cross-language and low-resource scenarios.

7.2.2 Input engineering

Input engineering focuses on optimizing the inputs to the model rather than altering its internal parameters. This category includes prompt tuning (Liu et al. 2022) and instruction tuning (Zhang et al. 2023).

Prompt tuning involves designing task-specific prompts that guide the PLMs to produce relevant outputs without significantly changing their parameters. This technique leverages the model's pre-existing knowledge and reframes tasks as prompt learning. Wang et al. (2022) applied prompt tuning to three downstream tasks in the field of software engineering, demonstrating its effectiveness in this domain. Li et al. (2023) utilized prompt tuning to predict the severity and exploitability features of vulnerabilities based on vulnerability descriptions. Lu et al. (2023) improved performance on vulnerability detection tasks using prompt tuning and demonstrated its effectiveness in cross-project and low-resource scenarios.

Instruction tuning extends the idea of prompt tuning by providing explicit task instructions to the model. This approach helps the model understand the specific requirements and constraints of the task. In software engineering, instruction tuning can enhance the performance of models on tasks like vulnerability detection by supplying precise instructions that highlight the critical aspects of code security. For

example, Yusuf and Jiang (2024) studied the impact of natural language instructions on improving the generalization ability of vulnerability detection tasks.

These fine-tuning approaches, whether through weight tuning or input engineering, underscore the versatility and adaptability of pre-trained models across various software engineering tasks. By incorporating structural knowledge and domain-specific information during the fine-tuning process, we can significantly enhance the performance of models in downstream tasks such as vulnerability detection.

7.3 Novelty of our study

Unlike existing vulnerability detection methods based on the pre-training and fine-tuning paradigm, we convert code datasets in plain text form into structural information during fine-tuning. We generate comprehensive and effective code structure information by using joern. This process generates a code structure matrix, and Sinkhorn Divergence is used to measure the discrepancy between the code structure matrix and the attention matrix, resulting in structural knowledge loss. During fine-tuning, we employ multi-task learning to jointly optimize both the structural knowledge loss and the objective function for vulnerability detection, thereby improving model performance. The synergy between structural and detection objectives leads to a more robust and effective vulnerability detection system, providing better performance and reliability in identifying and mitigating potential vulnerabilities in software.

8 Conclusion

Current research on vulnerability detection using PLMs still faces challenges in enhancing the structural knowledge of code during the fine-tuning phase (Jiang et al. 2024). To alleviate this issue, we propose SIFT, a novel multi-task fine-tuning approach designed to strengthen the domain-specific structural knowledge of PLMs during fine-tuning. We first generate the code property graph (CPG), and then, based on this, construct distance matrices and quantify the structural knowledge loss between attention matrices of Transformer blocks and these distance matrices using Sinkhorn divergence (Liu et al. 2022). By employing a multi-task learning strategy, SIFT enables the model to acquire code structure knowledge during fine-tuning, thereby enhancing vulnerability detection performance. Additionally, SIFT is a plug-and-play approach that can be used with any PLMs without altering the model architecture. Experimental results on three C/C++ datasets demonstrate that SIFT outperforms nine state-of-the-art vulnerability detection baselines, with F1 scores improving by 1.74%, 10.19%, and 2.87%, respectively. Furthermore, our experiments validate the contributions of various components within CPG and its effectiveness across different PLMs with different architectures and low-resource scenarios.

In future work, we aim to design more effective graph-based representation methods to enable models to learn richer structural knowledge (Qiu et al. 2024; Xiao et al. 2024). Additionally, we plan to conduct empirical studies on other programming languages. Lastly, we hope to enhance SIFT to address other challenges in the vulnerability domain, such as vulnerability repair (Zhou et al. 2024; Fu et al. 2022), which represents a valuable research direction.

Acknowledgements Liping Wang and Guilong Lu contributed equally to this work and are considered co-first authors. This research was supported by the National Natural Science Foundation of China (No. 61202006) and the University-Industry Collaborative Education Program (No. 220503175263849).

Author contributions Liping Wang: Data curation, Software, Validation, Conceptualization, Methodology, Writing—review & editing. Guilong Lu: Data curation, Software, Validation, Conceptualization, Methodology, Writing—review & editing. Xiang Chen: Conceptualization, Methodology, Writing—review & editing, Supervision. Xiaofeng Dai: Conceptualization, Writing—review & editing. Jianlin Qiu: Conceptualization, Writing—review & editing.

Funding National Natural Science Foundation of China (61202006).

Data availability No datasets were generated or analysed during the current study.

Declarations

Conflict of interest The authors declare no conflict of interest.

References

- Ahmad, W., Chakraborty, S., Ray, B., Chang, K.-W.: Unified pre-training for program understanding and generation. In: Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, pp. 2655–2668 (2021)
- Alon, U., Brody, S., Levy, O., Yahav, E.: code2seq: generating sequences from structured representations of code. In: International Conference on Learning Representations (2018)
- Alon, U., Zilberstein, M., Levy, O., Yahav, E.: code2vec: learning distributed representations of code. In: Proceedings of the ACM on Programming Languages, vol. 3(POPL), pp. 1–29 (2019)
- Aslan, Ö., Aktuğ, S.S., Ozkan-Okay, M., Yilmaz, A.A., Akin, E.: A comprehensive review of cyber security vulnerabilities, threats, attacks, and solutions. *Electronics* **12**(6), 1333 (2023)
- Ayupov, S., Chirkova, N.: Parameter-efficient finetuning of transformers for source code. *arXiv preprint arXiv:2212.05901* (2022)
- Behl, D., Handa, S., Arora, A.: A bug mining tool to identify and analyze security bugs using naive bayes and tf-idf. In: 2014 International Conference on Reliability Optimization and Information Technology (ICROIT), pp. 294–299. IEEE (2014)
- Cai, Z., Cai, Y., Chen, X., Lu, G., Pei, W., Zhao, J.: Csvd-tf: cross-project software vulnerability detection with tradaboost by fusing expert metrics and semantic metrics. *J. Syst. Softw.* **213**, 112038 (2024)
- Cao, S., Sun, X., Wu, X., Lo, D., Bo, L., Li, B., Liu, W.: Coca: improving and explaining graph neural network-based vulnerability detection systems. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, pp. 1–13 (2024)
- Chakraborty, S., Krishna, R., Ding, Y., Ray, B.: Deep learning based vulnerability detection: are we there yet? *IEEE Trans. Softw. Eng.* **48**(09), 3280–3296 (2022)
- Chen, D., Zhang, Y.-D., Wei, W., Wang, S.-X., Huang, R.-B., Li, X.-L., Qu, B.-B., Jiang, S.: Efficient vulnerability detection based on an optimized rule-checking static analysis technique. *Front. Inf. Technol. Electron. Eng.* **18**(3), 332–345 (2017)
- Chizat, L., Roussillon, P., Léger, F., Vialard, F.-X., Peyré, G.: Faster Wasserstein distance estimation with the Sinkhorn divergence. *Adv. Neural Inf. Process. Syst.* **33**, 2257–2269 (2020)

- Colon-Hernandez, P., Havasi, C., Alonso, J., Huggins, M., Breazeal, C.: Combining pre-trained language models and structured knowledge. arXiv preprint [arXiv:2101.12294](https://arxiv.org/abs/2101.12294) (2021)
- Fan, J., Li, Y., Wang, S., Nguyen, T.N.: Ac/c++ code vulnerability dataset with code changes and CVE summaries. In: Proceedings of the 17th International Conference on Mining Software Repositories, pp. 508–512 (2020)
- Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., et al.: Codebert: a pre-trained model for programming and natural languages. In: Findings of the Association for Computational Linguistics: EMNLP 2020, pp. 1536–1547 (2020)
- Fernandes, P., Allamanis, M., Brockschmidt, M.: Structured neural summarization. In: International Conference on Learning Representations (2018)
- Fu, M., Tantithamthavorn, C., Le, T., Nguyen, V., Phung, D.: Vulrepair: a t5-based automated software vulnerability repair. In: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 935–947 (2022)
- Fu, M., Tantithamthavorn, C.: Linevul: a transformer-based line-level vulnerability prediction. In: Proceedings of the 19th International Conference on Mining Software Repositories, pp. 608–620 (2022)
- Gonzalez, D., Hastings, H., Mirakhorli, M.: Automated characterization of software vulnerabilities. In: 2019 IEEE International Conference on Software Maintenance and Evolution (ICSME), pp. 135–139. IEEE (2019)
- Guo, D., Ren, S., Lu, S., Feng, Z., Tang, D., Shujie, L., Zhou, L., Duan, N., Svyatkovskiy, A., Fu, S., et al.: Graphcodebert: pre-training code representations with data flow. In: International Conference on Learning Representations (2020)
- Hadipour, N., Murimi, R.: Medical advisories as deterrents in healthcare cybercrime. In: 2024 Cyber Awareness and Research Symposium (CARS), pp. 1–6. IEEE (2024)
- Han, L., Erofeev, G., Sorokina, I., Gladkoff, S., Nenadic, G.: Examining large pre-trained language models for machine translation: what you don't know about it. In: Proceedings of the Seventh Conference on Machine Translation (WMT), pp. 908–919 (2022)
- Hanif, H., Maffei, S.: Vulberta: simplified source code pre-training for vulnerability detection. In: 2022 International Joint Conference on Neural Networks (IJCNN), pp. 1–8. IEEE (2022)
- Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., Wang, H.: Large language models for software engineering: a systematic literature review. *ACM Trans. Softw. Eng. Methodol.* **33**(8), 1–79 (2023)
- Hua, W., Sui, Y., Wan, Y., Liu, G., Xu, G.: FCCA: hybrid code representation for functional clone detection using attention networks. *IEEE Trans. Reliab.* **70**(1), 304–318 (2020)
- Jiang, Z., Sun, W., Gu, X., Wu, J., Wen, T., Hu, H., Yan, M.: Dfep: data flow embedding for enhancing pre-trained model based vulnerability detection. In: Proceedings of the 15th Asia-Pacific Symposium on Internetwork, pp. 95–104 (2024)
- Kalyan, K.S., Rajasekharan, A., Sangeetha, S.: Ammus: a survey of transformer-based pretrained models in natural language processing. arXiv preprint [arXiv:2108.05542](https://arxiv.org/abs/2108.05542) (2021)
- Kalyan, K.S.: A survey of GPT-3 family large language models including ChatGPT and GPT-4. *Nat. Lang. Process. J.* **6**, 100048 (2023)
- Karmakar, A., Robbes, R.: What do pre-trained code models know about code? In: 2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 1332–1336. IEEE (2021)
- Kim, S., Choi, J., Ahmed, M.E., Nepal, S., Kim, H.: Vuldebert: a vulnerability detection system using bert. In: 2022 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW), pp. 69–74. IEEE (2022)
- Lacombe, J., Digne, J., Courty, N., Bonneel, N.: Learning to generate Wasserstein Barycenters. *J. Math. Imaging Vis.* **65**(2), 354–370 (2023)
- Lamsiyah, S., Mahdaouy, A.E., Ouati, S.E.A., Espinasse, B.: Unsupervised extractive multi-document summarization method based on transfer learning from BERT multi-task fine-tuning. *J. Inf. Sci.* **49**(1), 164–182 (2023)
- Lewis, M., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A., Levy, O., Stoyanov, V., Zettlemoyer, L.: Bart: denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. arXiv preprint [arXiv:1910.13461](https://arxiv.org/abs/1910.13461) (2019)
- Li, C., Bi, B., Yan, M., Wang, W., Huang, S., Huang, F., Si, L.: Structurallm: structural pre-training for form understanding. In: Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing, vol. 1: Long Papers, pp. 6309–6318 (2021)

- Li, X., Ren, X., Xue, Y., Xing, Z., Sun, J.: Prediction of vulnerability characteristics based on vulnerability description and prompt learning. In: 2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), pp. 604–615. IEEE (2023)
- Li, Y., Wang, S., Nguyen, T.N.: Vulnerability detection with fine-grained interpretations. In: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 292–303 (2021)
- Li, Z., Zou, D., Xu, S., Ou, X., Jin, H., Wang, S., Deng, Z., Zhong, Y.: Vuldeepecker: a deep learning-based system for vulnerability detection. arXiv preprint [arXiv:1801.01681](https://arxiv.org/abs/1801.01681) (2018)
- Li, X., Wang, L., Xin, Y., Yang, Y., Chen, Y.: Automated vulnerability detection in source code using minimum intermediate representation learning. *Appl. Sci.* **10**(5), 1692 (2020)
- Li, Z., Zou, D., Xu, S., Jin, H., Zhu, Y., Chen, Z.: Sysevr: a framework for using deep learning to detect software vulnerabilities. *IEEE Trans. Dependable Secur. Comput.* **19**(4), 2244–2258 (2022)
- Li, Y., Zhang, T., Luo, X., Cai, H., Fang, S., Yuan, D.: Do pre-trained language models indeed understand software engineering tasks? *IEEE Trans. Softw. Eng.* **49**(10), 4639–4655 (2023)
- Liu, S., Chen, Y., Xie, X., Siow, J.K., Liu, Y.: Retrieval-augmented generation for code summarization via hybrid GNN. In: International Conference on Learning Representations (2020)
- Liu, X., Ji, K., Fu, Y., Tam, W., Du, Z., Yang, Z., Tang, J.: P-tuning: prompt tuning can be comparable to fine-tuning across scales and tasks. In: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics, vol. 2: Short Papers, pp. 61–68 (2022)
- Liu, F., Li, G., Zhao, Y., Jin, Z.: Multi-task learning based pre-trained language model for code completion. In: Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering, pp. 473–485 (2020)
- Liu, L., Huang, H., Gao, Y., Zhang, Y.: Improving neural topic modeling via Sinkhorn divergence. *Inf. Process. Manag.* **59**(3), 102864 (2022)
- Lu, S., Guo, D., Ren, S., Huang, J., Svyatkovskiy, A., Blanco, A., Clement, C., Drain, D., Jiang, D., Tang, D., et al.: Codexglue: a machine learning benchmark dataset for code understanding and generation. In: 35th Conference on Neural Information Processing Systems Datasets and Benchmarks Track (2021)
- Lu, G., Ju, X., Chen, X., Yang, S., Chen, L., Shen, H.: Assessing the effectiveness of vulnerability detection via prompt tuning: an empirical study. In: 2023 30th Asia-Pacific Software Engineering Conference (APSEC), pp. 415–424. IEEE (2023)
- Lu, G., Ju, X., Chen, X., Pei, W., Cai, Z.: GRACE: empowering LLM-based software vulnerability detection with graph structure and in-context learning. *J. Syst. Softw.* **212**, 112031 (2024)
- Nguyen, V.-A., Nguyen, D.Q., Nguyen, V., Le, T., Tran, Q.H., Phung, D.: Regvd: revisiting graph neural networks for vulnerability detection. In: Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings, pp. 178–182 (2022)
- Pham, N.H., Nguyen, T.T., Nguyen, H.A., Nguyen, T.N.: Detection of recurring software vulnerabilities. In: Proceedings of the 25th IEEE/ACM International Conference on Automated Software Engineering, pp. 447–456 (2010)
- Qiu, F., Liu, Z., Hu, X., Xia, X., Chen, G., Wang, X.: Vulnerability detection via multiple-graph-based code representation. *IEEE Trans. Softw. Eng.* (2024). <https://doi.org/10.1109/TSE.2024.3427815>
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., Liu, P.J.: Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.* **21**(140), 1–67 (2020)
- Russell, R., Kim, L., Hamilton, L., Lazovich, T., Harer, J., Ozdemir, O., Ellingwood, P., McConley, M.: Automated vulnerability detection in source code using deep representation learning. In: 2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA), pp. 757–762. IEEE (2018)
- Sejfić, A., Das, S., Shafiq, S., Medvidović, N.: toward improved deep learning-based vulnerability detection. In: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, pp. 1–12 (2024)
- Siow, J.K., Liu, S., Xie, X., Meng, G., Liu, Y.: Learning program semantics with code representations: An empirical study. In: 2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), pp. 554–565. IEEE (2022)
- Tian, H., Liu, K., Kaboré, A.K., Koyuncu, A., Li, L., Klein, J., Bissyandé, T.F.: Evaluating representation learning of code changes for predicting patch correctness in program repair. In: Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering, pp. 981–992 (2020)

- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. In: *Advances in Neural Information Processing Systems*, vol 30 (2017)
- Wang, D., Chen, B., Li, S., Luo, W., Peng, S., Dong, W., Liao, X.: One adapter for all programming languages? Adapter tuning for code search and summarization. In: *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pp. 5–16. IEEE (2023)
- Wang, W., Li, G., Ma, B., Xia, X., Jin, Z.: Detecting code clones with graph neural network and flow-augmented abstract syntax tree. In: *2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pp. 261–271. IEEE (2020)
- Wang, Y., Wang, W., Joty, S., Hoi, S.C.: Codet5: identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 8696–8708 (2021)
- Wang, C., Yang, Y., Gao, C., Peng, Y., Zhang, H., Lyu, M.R.: No more fine-tuning? An experimental evaluation of prompt tuning in code intelligence. In: *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pp. 382–394 (2022)
- Wang, H., Ye, G., Tang, Z., Tan, S.H., Huang, S., Fang, D., Feng, Y., Bian, L., Wang, Z.: Combining graph-based learning with automated data collection for code vulnerability detection. *IEEE Trans. Inf. Forensics Secur.* **16**, 1943–1958 (2020)
- Wen, X.-C., Chen, Y., Gao, C., Zhang, H., Zhang, J.M., Liao, Q.: Vulnerability detection with graph simplification and enhanced graph representation learning. In: *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pp. 2275–2286. IEEE (2023)
- Weng, L.: From gan to wgan. arXiv preprint [arXiv:1904.08994](https://arxiv.org/abs/1904.08994) (2019)
- Wilcoxon, F.: Individual comparisons by ranking methods. In: *Breakthroughs in Statistics: Methodology and Distribution*, pp. 196–202. Springer, Cham (1992)
- Xiao, P., Xiao, Q., Zhang, X., Wu, Y., Yang, F.: Vulnerability detection based on enhanced graph representation learning. *IEEE Trans. Inf. Forensics Secur.* (2024). <https://doi.org/10.1109/TIFS.2024.3392536>
- Yusuf, I.N.B., Jiang, L.: Your instructions are not always helpful: assessing the efficacy of instruction fine-tuning for software vulnerability detection. arXiv preprint [arXiv:2401.07466](https://arxiv.org/abs/2401.07466) (2024)
- Zhang, S., Dong, L., Li, X., Zhang, S., Sun, X., Wang, S., Li, J., Hu, R., Zhang, T., Wu, F., et al.: Instruction tuning for large language models: a survey. arXiv preprint [arXiv:2308.10792](https://arxiv.org/abs/2308.10792) (2023)
- Zhang, R., Han, J., Liu, C., Gao, P., Zhou, A., Hu, X., Yan, S., Lu, P., Li, H., Qiao, Y.: Llama-adapter: efficient fine-tuning of language models with zero-init attention. arXiv preprint [arXiv:2303.16199](https://arxiv.org/abs/2303.16199) (2023)
- Zhang, J., Wang, X., Zhang, H., Sun, H., Wang, K., Liu, X.: A novel neural source code representation based on abstract syntax tree. In: *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, pp. 783–794. IEEE (2019)
- Zheng, Z., Ning, K., Chen, J., Wang, Y., Chen, W., Guo, L., Wang, W.: Towards an understanding of large language models in software engineering tasks. arXiv preprint [arXiv:2308.11396](https://arxiv.org/abs/2308.11396) (2023)
- Zhou, X., Cao, S., Sun, X., Lo, D.: Large language model for vulnerability detection and repair: literature review and roadmap. arXiv preprint [arXiv:2404.02525](https://arxiv.org/abs/2404.02525) (2024)
- Zhou, Y., Liu, S., Siow, J., Du, X., Liu, Y.: Devign: effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In: *Advances in Neural Information Processing Systems*, vol. 32 (2019)
- Zou, D., Wang, S., Xu, S., Li, Z., Jin, H.: μ vuldeepecker: a deep learning-based system for multiclass vulnerability detection. *IEEE Trans. Dependable Secur. Comput.* **18**(5), 2224–2236 (2019)
- Zugner, D., Kirschstein, T., Catasta, M., Leskovec, J., Gunnemann, S.: Language-agnostic representation learning of source code from structure and context. In: *International Conference on Learning Representations (ICLR)* (2021)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Liping Wang¹ · Guilong Lu² · Xiang Chen² · Xiaofeng Dai¹ · Jianlin Qiu²

✉ Xiang Chen
xchencs@ntu.edu.cn

Liping Wang
nieyuwang@outlook.com

Guilong Lu
guil.lu@outlook.com

Xiaofeng Dai
xiaofdai@outlook.com

Jianlin Qiu
jianlqiu@outlook.com

¹ School of Yonyou Digitaland Intelligence, Nantong Institute of Technology, Nantong 226002, Jiangsu, China

² School of Artificial Intelligence and Computer Science, Nantong University, Nantong 226019, Jiangsu, China